

OSP: Joint Access to Multiple Nodes

Named cursors · Joint entry events · One integer per node

Problem

A dot product reads two sequences together. An ordinary rhs reference hides the second traversal channel.

Solution

Declare a fixed set of named channels. Execute an ability only when all current nodes match its entry predicates.

Current goal: correct execution on the Python CPU backend. GPU layout and address optimization are future work.

One Output, Two Input Paths

$$C[i,j] = \sum A[i,k] \times B[k,j]$$

A row i: 1→2→3

B column j: 7→9→11

$$k=0: 1 \times 7 \quad k=1: 2 \times 9 \quad k=2: 3 \times 11$$

$$C[i,j] = 7 + 18 + 33 = 58$$

Each step consumes a corresponding pair of nodes. Each input node stores one integer.

Existing Pattern: here Plus an rhs Reference

```
self.total += here.value * self.rhs.value;  
following = [self.rhs->:ColumnNext:->];  
if following { self.rhs = following[0]; }  
visit [->:RowNext:->];
```

Explicit primary traversal

The walker queue and visit manage the movement of here.

Secondary traversal in private state

The programmer maintains rhs and its pairing manually. The compiler must infer its access pattern.

A regular column chain can be deterministic. This expression alone does not guarantee physical layout or fixed-stride addressing.

Make Both Positions First-Class Channels

```
walker Dot {  
  cursor a, b;  
  has total: int = 0;  
  
  can multiply with (a: Scalar entry, b: Scalar entry) {  
    self.total += here[a].value * here[b].value;  
    visit[a] [->:RowNext:->];  
    visit[b] [->:ColumnNext:->];  
  }  
}
```

Cursor declarations are untyped. The programmer names the channels. Both entry predicates may match the same Scalar type.

One Joint Event, Two Independent Queues

Channel a / RowNext

Current node: 1. Schedule successor: 2.

Channel b / ColumnNext

Current node: 7. Schedule successor: 9.

Round 1: a=1, b=7 → multiply runs once

Both positions stay stable while abilities execute.

visit schedules arrivals; it does not move the current nodes.

Round 2: a=2, b=9 → match the joint predicates again

All predicates must match. multiply never observes one channel halfway through its advancement.

Preserve here; Define the New Event Contract

here[a] is channel a's current node. Implicit edge origins inside visit[a] use that node. Legacy walkers keep their original execution path.

Round start: consume one fresh arrival per channel

Multiple targets: FIFO pairing; no Cartesian product

Channel exhausted: end traversal; discard unmatched tails

No visit: consume queued siblings, or become exhausted

Old entry: never reused as a new arrival

These are the choices recorded in the CPU design document. Joint entry is an arrival-event barrier, not repeated triggering on a held node.

CPU Matmul: Shared Scalar Nodes

```
node Scalar { has value: int; }
```

```
edge RowNext {}
```

```
edge ColumnNext {}
```

```
# Launch one Dot for each C[i,j].
```

```
w = Dot();
```

```
{"a": A_row_head[i], "b": B_col_head[j]} spawn w;
```

```
# Write w.total into the Scalar node for C[i,j].
```

Shared inputs

MK nodes for A and KN nodes for B. Output walkers share those inputs rather than copying each multiplication pair.

Independent results

MN scalar output nodes. Each output independently accumulates K products.

Illustrative caller code. Graph construction, bindings and output writes stay on the host.

Predictable Traversal vs. Predictable Addresses

What the semantics expose

Which nodes are accessed together, which relations advance them, and when the joint ability fires.

What layout optimization still needs

Shape, stride, regular adjacency, access permissions and a suitable physical representation.

Candidate lowering for a regular row-major layout:

A address = $\text{base_A} + (i * K + k) \times \text{sizeof}(\text{int})$

B address = $\text{base_B} + (k * N + j) \times \text{sizeof}(\text{int})$

CPU named cursors do not guarantee contiguous storage, cache hits or fixed latency. Arbitrary graph successors may still require edge-data loads.

Beyond Matmul: Joint Multi-Input Computation

Dot product / $\alpha x + y$: two input sequences, paired steps

Strassen sums: corresponding nodes of two quadrants

Strassen recombine: corresponding nodes of P1, P4, P5, P7

$$C00 = P1 + P4 - P5 + P7$$

Recursive task scheduling, dependencies and temporary storage remain separate concerns.

Sparse intersection needs one-sided advancement while retaining the other position. The current fresh-arrival contract cannot directly provide that behavior. Stencils also need boundary rules.

CPU Correctness First, Layout Optimization Next

Implementation scope

Parser / AST / semantic checks

Python code generation and runtime

Dot product and scalar-node matmul

Boundary tests and legacy compatibility

Key validation cases

Same-type and heterogeneous matching

FIFO branches, empty and unequal queues

Stable current positions and correct outputs

Explicit rejection of unsupported combinations

Design value: make joint access relationships explicit in OSP.

Sources: our design discussion, docs/design/named-cursors.md, and the dual-cursor example in graphs.jac.
Implementation is in progress; this deck claims no completed test results.